

map, filter e reduce

COMP0496 – Programação A

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

andreyoshiaki@dcomp.ufs.br

Ao trabalhar com listas, três operações aparecem repetidamente:

1. **Transformar** cada elemento: *“converter todas as notas de 0–10 para 0–100”*
2. **Selecionar** alguns elementos: *“manter só os aprovados”*
3. **Combinar** todos em um único resultado: *“calcular o produto de todos os elementos”*

Python oferece funções dedicadas para cada padrão: `map`, `filter` e `reduce`.

map

`map(f, seq)` aplica a função `f` a cada elemento de `seq`. Retorna um *iterador*— use `list()` para materializar.

```
1  palavras = ["python", "e", "divertido"]
2  maiusculas = list(map(str.upper, palavras))
3  print(maiusculas)    # ['PYTHON', 'E', 'DIVERTIDO']
4
5  notas = [6.0, 7.5, 8.0, 9.5]
6  # Converter notas de 0-10 para 0-100
7  em_100 = list(map(lambda n: n * 10, notas))
8  print(em_100)       # [60.0, 75.0, 80.0, 95.0]
```

filter

`filter(pred, seq)` mantém apenas os elementos onde `pred(x)` é `True`. Também retorna um iterador.

```
1 notas = [3.5, 7.0, 4.8, 9.2, 5.0, 2.1]
2
3 aprovados = list(filter(lambda n: n >= 5.0, notas))
4 print(aprovados)      # [7.0, 9.2, 5.0]
5
6 # Equivalente com compreensao:
7 aprovados2 = [n for n in notas if n >= 5.0]
8 print(aprovados2)     # [7.0, 9.2, 5.0]
```

```
1  from functools import reduce
2
3  numeros = [1, 2, 3, 4, 5]
4
5  # Soma acumulada: (((1+2)+3)+4)+5 = 15
6  soma = reduce(lambda acc, x: acc + x, numeros)
7  print(soma)      # 15
8
9  # Produto: 1*2*3*4*5 = 120
10 produto = reduce(lambda acc, x: acc * x, numeros)
11 print(produto)   # 120
12
13 # Maximo (sem usar max()):
14 maximo = reduce(lambda a, b: a if a > b else b, numeros)
15 print(maximo)    # 5
```

reduce aplica $f(\text{acc}, x)$ acumulando da esquerda para a direita.

Problema: dado uma lista de notas, filtrar aprovados e calcular a média.

```
1  from functools import reduce
2
3  notas = [3.5, 7.0, 4.8, 9.2, 5.0, 2.1]
4
5  aprovados = list(filter(lambda n: n >= 5.0, notas))
6  # [7.0, 9.2, 5.0]
7
8  soma = reduce(lambda acc, x: acc + x, aprovados)
9  media = soma / len(aprovados)
10 print(f"Media dos aprovados: {media:.2f}") # 7.07
11
12 # Equivalente com compreensao (mais legivel):
13 ap = [n for n in notas if n >= 5.0]
14 media2 = sum(ap) / len(ap)
```

Critério	map/filter	Compreensão
Legibilidade	Média	Alta
Composição com funções	Direta	Requer lambda
Avaliação	Lazy (iterador)	Eager (lista)
Estilo Pythônico	Menos comum	Preferido

Regra prática: prefira compreensões para legibilidade; use `map/filter` quando compuser com outras funções de alta ordem ou precisar de avaliação lazy.

- `map(f, seq)` transforma cada elemento aplicando `f`.
- `filter(pred, seq)` seleciona elementos onde `pred(x)` é verdadeiro.
- `reduce(f, seq)` acumula todos os elementos em um único resultado.
- `map` e `filter` retornam *iteradores* — use `list()` para obter uma lista.